

Programming for AI

AI-2001

Dictionaries

National University of Computer & Emerging Sciences – NUCES – Karachi



National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2001

Programming for AI

Contents

1. Introduction	2
2. Dictionaries	2
2.1. Unique Keys	2
2.2. Creating a Dictionary	2
2.2. Determining if a Dictionary is Empty	3
2.3. Iterating through a Dictionaries	3
3. Basic Dictionaries Operations	4
3.1. Accessing the Value Associated with a Key	4
3.2. Updating the Value of an Existing Key–Value Pair	4
3.3. Adding a New Key–Value Pair	5
3.4. Removing a Key–Value Pair	5
3.5. Attempting to Access a Non-existent Key	5
3.6. Testing Whether a Dictionary Contains a Specified Key	6
4. Dictionary Methods keys and values	6
4.1. Dictionary Views	6
4.2 Converting Dictionary Keys, Values and Key–Value Pairs to Lists	7
4.3 Processing Keys in Sorted Order	7
5. Example: Dictionary of Student Grades	7

1. Introduction

A dictionary is an unordered collection that stores key-value pairs, where immutable keys are mapped to corresponding values, similar to how a traditional dictionary links words to their definitions. A set, on the other hand, is an unordered collection of unique, immutable elements.

2. Dictionaries

A dictionary links keys to values, with each key corresponding to a particular value. The table below provides examples of dictionaries, showing their keys, key types, values, and value types.

Key	Key Type	Values	Value Type
Country Names	str	Internet Country Codes	str
Decimal Numbers	int	Roman Numerals	str
States	str	Agricultural Products	list of str
Hospital Patients	str	Vital Signs	tuple of ints and floats
Baseball Players	str	Batting Averages	float
Metric Measurements	str	Abbreviations	str
Inventory Codes	str	Quantity in Stock	int

2.1. Unique Keys

The keys in a dictionary must be immutable (like strings, numbers, or tuples) and must be unique (no duplicates are allowed). However, different keys can share the same value, such as two distinct inventory codes having the same stock quantity.

2.2. Creating a Dictionary

You can create a dictionary by placing a comma-separated list of key-value pairs inside curly braces {}, where each pair follows the format key: value. An empty dictionary can be created using just {}. For example, let's create a dictionary with the country names 'Finland', 'South Africa', and 'Nepal' as keys, and their corresponding Internet country codes 'fi', 'za', and 'np' as values.

[1]	<code>country_codes = {'Finland': 'fi', 'South Africa': 'za', 'Nepal': 'np'}</code>
[2]	<code>country_codes</code>

```
{'Finland': 'fi', 'South Africa': 'za', 'Nepal': 'np'}
```

When a dictionary is displayed, its key-value pairs are shown as a comma-separated list enclosed in curly braces. Since dictionaries are unordered collections, the display order may not match the order in which the key-value pairs were added. In the output of snippet [2], the key-value pairs appear in the order of insertion, but it's important not to rely on this ordering in your code.

2.2. Determining if a Dictionary is Empty

The built-in function `len` returns the number of key-value pairs in a dictionary:

```
[3] len(country_codes)
```

3

A dictionary can be used in a conditional statement to check if it's empty—any non-empty dictionary will evaluate as `True`.

```
[4] if country_codes:
    print('country_codes is not empty')
else:
    print('country_codes is empty')
```

country_codes is not empty

An empty dictionary evaluates as `False`. To illustrate this, the following code uses the `clear` method to remove all key-value pairs from the dictionary, and then snippet [6] recalls and re-executes snippet [4].

```
[5] country_codes.clear()
```

```
[6] if country_codes:
    print('country_codes is not empty')
else:
    print('country_codes is empty')
```

country_codes empty

2.3. Iterating through a Dictionaries

The dictionary below associates month names (as strings) with integer values representing the number of days in each month. It's important to note that different keys can share the same value.

```
[1] days_per_month = {'January': 31, 'February': 28, 'March': 31}
```

```
[2] days_per_month
{'January': 31, 'February': 28, 'March': 31}
```


Once again, the dictionary's string representation displays key-value pairs in the order they were inserted, though this is not guaranteed since dictionaries are unordered. Later in this chapter, we'll demonstrate how to process keys in sorted order. The following for loop iterates over the key-value pairs in `days_per_month`. The `items` method of the dictionary returns each pair as a tuple, which is then unpacked into `month` and `days`.

```
[3] for month, days in days_per_month.items():  
    print(f'{month} has {days} days')
```

```
January has 31 days  
February has 28 days  
March has 31 days
```

3. Basic Dictionaries Operations

Let's start by creating and displaying the `roman_numerals` dictionary. We deliberately use the incorrect value of 100 for the key 'X', which we will correct shortly.

```
[1] roman_numerals = {'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 100}
```

```
[2] roman_numerals  
roman_numerals
```

Talha Shahid

3.1. Accessing the Value Associated with a Key

Let's get the value associated with the key 'V':

```
[3] roman_numerals['V']  
5
```

3.2. Updating the Value of an Existing Key–Value Pair

You can modify a key's value using an assignment statement, as demonstrated here to correct the incorrect value associated with the key 'X'.

```
[4] roman_numerals['X'] = 10
```

```
[5] roman_numerals  
{'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 10}
```

3.3. Adding a New Key–Value Pair

Assigning a value to a non-existent key inserts the key–value pair in the dictionary:

```
[6] | roman_numerals['L'] = 50
```

```
[7] | roman_numerals
```

```
{'I': 1, 'II': 2, 'III': 3, 'V': 5, 'X': 10, 'L': 50}
```

String keys are case-sensitive. Assigning a value to a key that doesn't exist will create a new key–value pair. This behaviour might be intentional, but it could also indicate a logic error.

3.4. Removing a Key–Value Pair

You can delete a key–value pair from a dictionary with the `del` statement:

```
[8] | del roman_numerals['III']
```

```
[9] | roman_numerals
```

```
{'I': 1, 'II': 2, 'V': 5, 'X': 10, 'L': 50}
```

You also can remove a key–value pair with the dictionary method `pop`, which returns the value for the removed key:

```
[10] | roman_numerals.pop('X')
```

```
[11] | roman_numerals
```

```
{'I': 1, 'II': 2, 'V': 5, 'L': 50}
```

3.5. Attempting to Access a Non-existent Key

Accessing a non-existent key results in a `KeyError`:

```
[12] | roman_numerals['III']
```

```
-----
```

```
KeyError                                Traceback (most recent call last)
```

```
Cell In[21], line 1
```

```
----> 1 roman_numerals['III']
```

```
KeyError: 'III'
```

You can avoid this error by using the dictionary method `get`, which typically returns the value associated with the specified key. If the key is not present, `get` returns `None`. In Python, nothing is displayed if `None` is returned, as shown in snippet [13]. However, if you provide a second argument to `get`, it will return that value if the key is not found.

[13]	<code>roman_numerals.get('III')</code>
[14]	<code>roman_numerals.get('III', 'III not in dictionary')</code> <code>'III not in dictionary'</code>
[15]	<code>roman_numerals.get('V')</code>

5

3.6. Testing Whether a Dictionary Contains a Specified Key

Operators `in` and `not in` can determine whether a dictionary contains a specified key:

[16]	<code>'V' in roman_numerals</code>
	<code>True</code>
[17]	<code>'III' in roman_numerals</code>
	<code>False</code>
[18]	<code>'III' not in roman_numerals</code>
	<code>True</code>

4. Dictionary Methods keys and values

Previously, we used the dictionary method `items` to iterate through tuples of key-value pairs. Similarly, the methods `keys` and `values` can be used to iterate through just the dictionary's keys or values, respectively.

[1]	<code>months = {'January': 1, 'February': 2, 'March': 3}</code>
[2]	<code>for month_name in months.keys():</code> <code>print(month_name, end=' ')</code>
	<code>January February March</code>
[3]	<code>for month_number in months.values():</code> <code>print(month_number, end=' ')</code>
	<code>1 2 3</code>

4.1. Dictionary Views

The dictionary methods `items`, `keys`, and `values` each return a view of the dictionary's data. When you iterate over a view, it reflects the current state of the dictionary's contents and does not maintain a separate copy of the data.

To demonstrate that views do not keep their own copies, let's first assign the view returned by `keys` to the variable `months_view`, and then iterate through it.

[4]	<code>months_view = months.keys()</code>
[5]	<code>for key in months_view:</code> <code>print(key, end=' ')</code>
	<code>January February March</code>

Next, let's add a new key-value pair to `months` and display the updated dictionary:

[6]	<code>months['December'] = 12</code>
[7]	<code>months</code> <code>{'January': 1, 'February': 2, 'March': 3, 'December': 12}</code>

Now, let's iterate through `months_view` again. The key we added above is indeed displayed:

[8]	<code>for key in months_view:</code> <code> print(key, end=' ')</code> <code>January February March December</code>
-----	--

Avoid modifying a dictionary while iterating over a view. According to Section 4.10.1 of the Python Standard Library documentation,¹ this could either result in a `RuntimeError` or cause the loop to skip some values in the view.

4.2 Converting Dictionary Keys, Values and Key–Value Pairs to Lists

At times, you may need lists of a dictionary's keys, values, or key-value pairs. To create these lists, pass the view returned by `keys`, `values`, or `items` to the built-in list function. Modifying these lists will not affect the original dictionary.

[9]	<code>list(months.keys())</code> <code>['January', 'February', 'March', 'December']</code>
[10]	<code>list(months.values())</code> <code>[1, 2, 3, 12]</code>
[11]	<code>list(months.items())</code> <code>[('January', 1), ('February', 2), ('March', 3), ('December', 12)]</code>

4.3 Processing Keys in Sorted Order

To process keys in sorted order, you can use built-in function `sorted` as follows:

[12]	<code>for month_name in sorted(months.keys()):</code> <code> print(month_name, end=' ')</code> <code>December February January March</code>
------	--

5. Example: Dictionary of Student Grades

The script below models an instructor's grade book as a dictionary, where each student's name (a string) is mapped to a list of integers representing their grades from three exams. In each loop iteration that displays the data (lines 13–17), a key-value pair is unpacked into the variables `name` and `grades`, which hold a student's name and their list of three grades. Line 14 uses the built-in `sum` function to calculate the total of a student's grades, and line 15 computes and displays their average by dividing the total by the number of grades (`len(grades)`). Lines 16–17 track the total grades for all four students and the number of grades for everyone. Finally, line 19 prints the class average for all students across all exams.


```
01:
02:
03: grade_book = {
04:     'Rock': [92, 85, 100],
05:     'John': [83, 95, 79],
06:     'Mark': [91, 89, 82],
07:     'Mike': [97, 91, 92]
08: }
09:
10: all_grades_total = 0
11: all_grades_count = 0
12:
13: for name, grades in grade_book.items():
14:     total = sum(grades)
15:     print(f'Average for {name} is {total/len(grades):.2f}')
16:     all_grades_total += total
17:     all_grades_count += len(grades)
18:
19: print(f"Class's average is: {all_grades_total / all_grades_count:.2f}")
```

Talha Shahid